SR UNIVERSITY

AI Assistant Coding
Assignment-10.3

Name of Student : V. Abhinav Batch : 41
Enrollment No. : 2303A52486

Task 1: AI-Assisted Bug Detection

```
def factorial(n):
    result = 1

    for i in range(1, n):
        result = result * i
    return result
```

Original Code:

```
Assign 10.3.py > ...
def factorial(n):
    result = 1
    for i in range(1, n):
        result = result * i
    return result

print(factorial(5))
```

Output:

```
Problems   Output   Debug Console   Terminal   Ports   Python
PS C:\Users\Abhi\Documents\AI Assistant Coding> & C:/Users/Abhi/AppD
thon/Python314/python.exe "c:/Users/Abhi/Documents/AI Assistant Codi
24
```

Prompt:

Check if the above code contains any bugs and if yes then fix them. And Explain what the bug occured.

AI Corrected code:

```
# To fix the bug, we can change the range function to range(1, n+1).
# The fixed code is:
def factorial(n):
    result = 1
    for i in range(1, n+1):
        result = result * i
    return result

print(factorial(5))
```

Output:

```
PS C:\Users\Abhi\Documents\AI Assistant Coding> &
thon/Python314/python.exe "c:/Users/Abhi/Document
120
```

Explanation:

This task checks whether the factorial function is correct and uses AI to detect logical errors in the loop range. The original code used `range(1, n)`, which excluded `n`, so `factorial(5)` incorrectly produced **24** instead of **120**, as seen in the output on page 1. AI fixed it by changing the loop to `range(1, n+1)`, producing the correct result. However, the AI did miss some edge cases such as handling **negative numbers, zero, and non-integer inputs**, which would require validation logic. This shows AI is effective for syntax/logic fixes but still needs human oversight for robustness.

Task 2: Task 2 — Improving Readability & Documentation

```
def calc(a, b, c):
    if c == "add":
        return a + b
    elif c == "sub":
        return a - b
    elif c == "mul":
        return a * b
    elif c == "div": return a/b
```

Original Code Output:

```
PS C:\Users\Abhi\Documents\AI Assistant Coding> & C
s/Abhi/Documents/AI Assistant Coding/Assign 10.3.py
30
```

Prompt:

The above code function's readability, parameter naming, and lack of documentation. Rewrite the function with:

1. *Readable parameter names*
2. *Documentation*
3. *Error handling*
4. *Performance optimization*

Also provide a few unit tests,

Code with AI Improvements:

Assign 10.3.py > ...

```python
def calculate(num1, num2, operation):
    """
    This function performs basic arithmetic operations on two numbers.

    Parameters:
    num1 (float): The first number.
    num2 (float): The second number.
    operation (str): The operation to perform. It can be "add", "subtract", "multiply"

    Returns:
    float: The result of the arithmetic operation.

    Raises:
    ValueError: If the operation is not one of the specified operations.
    ZeroDivisionError: If the operation is division and num2 is zero.
    """

    if operation == "add":
        return num1 + num2
    elif operation == "subtract":
        return num1 - num2
    elif operation == "multiply":
        return num1 * num2
    elif operation == "divide":
        if num2 == 0:
            raise ZeroDivisionError("Cannot divide by zero.")
        return num1 / num2
    else:
        raise ValueError("Invalid operation. Please choose from 'add', 'subtract', 'mu
```

```python
# Unit tests
def test_calculate():
    assert calculate(10, 20, "add") == 30
    assert calculate(10, 20, "subtract") == -10
    assert calculate(10, 20, "multiply") == 200
    assert calculate(10, 20, "divide") == 0.5
    try:
        calculate(10, 0, "divide")
    except ZeroDivisionError:
        pass
    else:
        assert False, "Expected ZeroDivisionError"

    try:
        calculate(10, 20, "modulus")
    except ValueError:
        pass
    else:
        assert False, "Expected ValueError"
    print("All tests passed.")

if __name__ == "__main__":
    print(calculate(10, 20, "add"))
    print(calculate(10, 20, "subtract"))
    print(calculate(10, 20, "multiply"))
    print(calculate(10, 20, "divide"))
    test_calculate()
```

Output:

```
exe "c:/Users/Abhi/Documents/AI Assistant Coding/Assign 10.3.py"
30
-10
200
0.5
All tests passed.
```

Explanation:

This task improves a calculator function by replacing unclear parameter names, adding documentation, error handling, and tests. The AI rewrote the function using descriptive names, added docstrings, handled invalid operations, and prevented division by zero. The output confirms correctness by printing valid results and showing that all unit tests passed. This demonstrates how AI can significantly improve maintainability and reliability without changing the core logic. The improved structure also makes the function easier to reuse in larger systems.

Task 3: Enforcing Coding Standards

Scenario: A team project requires PEP8 compliance. A developer submits:

```
def Checkprime(n):
        for i in range(2, n):
            if n % i == 0:
                return False
        return True
```

Original Code Output:

```
exe "c:/Users/Abhi/Documents/AI Assistant Co
Enter a number: 11
True
```

Prompt:

Does the above code violate PEP-8 standards? If yes then list all the violations and fix them.

AI Corrected Code:

```
# Yes, the above code violates PEP-8 standards. The violations are:
# 1. The function name should be in lowercase with words separated by underscores. So, Checkprime should be changed to check_prime.
# 2. There should be a space after the comma in the input function. So,
# 3. The variable name n should be more descriptive. So, it can be changed to number.
# The fixed code is:
def check_prime(number):
    for i in range(2, number):
        if number % i == 0:
            return False
    return True
number = int(input("Enter a number: "))
print(check_prime(number))
```

Output:

```
PS C:\Users\Abhi\Documents\AI Assistant Coding> & C:/Users/Abhi/A
Enter a number: 12
False
Enter a number: 11
True
```

Explanation:

This task checks whether a prime-checking function follows PEP-8 coding standards and uses AI to identify violations. The AI corrected issues such as improper naming (`Checkprime` → `check_prime`), poor indentation, and unclear variable naming while preserving the logic, and the output confirms correct behavior for inputs like 11 and 12. Automated AI reviews like this can streamline large-team workflows by quickly flagging style issues, enforcing consistency, and reducing manual review time. They also allow human reviewers to focus on design, logic, and security rather than formatting problems. Thus, AI can act as a first-pass reviewer to speed up collaborative development.

Task 4: AI as a Code Reviewer in Real Projects

Scenario:
In a GitHub project, a teammate submits:

```
def processData(d):
    return [x * 2 for x in d if x % 2 == 0]
```

Original Code Output:

```
PS C:\Users\Abhi\Documents\AI Assistant Coding> & C:/Users/Abhi
[4, 8, 12]
```

Prompt:
Generate a code review for the above code:

a. Better naming and function purpose clarity.

b. Input validation and type hints.

c. Suggestions for generalization (e.g., configurable multiplier).

AI Code Review:

A. Better naming and function purpose clarity:
   a. The function name "processData" is not descriptive of what the function does. A better name could be "double_even_numbers" to

clearly indicate that the function doubles the even numbers in the input list.

b. The variable name "d" is not descriptive. A better name could be "numbers" to indicate that it is a list of numbers.

B. Input validation and type hints:

a. The function does not currently validate the input. It would be beneficial to add input validation to ensure that the input is a list of integers.

b. Adding type hints can improve code readability and help with debugging. For example, the function signature could be updated to:

c. ``def double_even_numbers(numbers: list[int]) -> list[int]:

C. Suggestions for generalization (e.g., configurable multiplier):

a. The function currently doubles the even numbers. It would be more flexible to allow the multiplier to be configurable. This can be achieved by adding an additional parameter for the multiplier.

AI Code with Improvements:

```python
# Regenerated code with improvements:
def double_even_numbers(numbers: list[int], multiplier: int = 2) -> list[int]:
    """
    This function takes a list of integers and returns a new list containing the even numbers from the input list multiplied by a specified multiplier.

    Parameters:
    numbers (list[int]): A list of integers to be processed.
    multiplier (int): The value by which to multiply the even numbers. Default is 2.

    Returns:
    list[int]: A list of even numbers from the input list, each multiplied by the specified multiplier.

    Raises:
    ValueError: If the input is not a list of integers.
    """
    if not isinstance(numbers, list) or not all(isinstance(x, int) for x in numbers):
        raise ValueError("Input must be a list of integers.")

    return [x * multiplier for x in numbers if x % 2 == 0]
data = [1, 2, 3, 4, 5, 6]
print(double_even_numbers(data))
```

Output:

```
PS C:\Users\Abhi\Documents\AI Assistant Coding> & C:/Users/Abh
[4, 8, 12]
PS C:\Users\Abhi\Documents\AI Assistant Coding>
```

Explanation:

This task demonstrates AI generating a structured code review for a function that doubles even numbers in a list. The AI suggested better naming, input validation, type hints, and a configurable multiplier, then produced an improved version whose output correctly returns `[4, 8, 12]`. This shows AI's usefulness in identifying clarity, flexibility, and maintainability improvements beyond simple bug fixing. However, AI should function as an **assistant rather than a standalone reviewer**, since human developers are still needed to judge architectural decisions, context, and business requirements. The best approach is collaborative review where AI speeds analysis and humans make final decisions.

Task 5: AI-Assisted Performance Optimization

Scenario: You are given a function that processes a list of integers, but it runs slowly on large datasets:

```
def sum_of_squares(numbers):
    total = 0
    for num in numbers:
    total += num ** 2
    return total
```

Original Code Output:

```
PS C:\Users\Abhi\Documents\AI Assistant Coding> & C:/Us
55
PS C:\Users\Abhi\Documents\AI Assistant Coding>
```

Prompt:
Test the above function with a large list of numbers to check its performance.

- Analyze time complexity.
- Suggest performance improvements (e.g., using built-in functions, vectorization with NumPy if applicable).

AI Answers:

- Time Complexity:
  - The time complexity of the sum_of_squares function is O(n), where n is the number of elements in the input list "numbers".
  - This is because the function iterates through each element in the list exactly once to calculate the sum of squares.
- Performance Improvements:
  - Using built-in functions: We can use the built-in sum function along with a generator expression to calculate the sum of squares more efficiently.
  - This can reduce the overhead of manually iterating through the list and updating the total variable.

Optimized Code by AI:

```
# Provide an optimized version.
def sum_of_squares_optimized(numbers):
    return sum(x ** 2 for x in numbers)

numbers = list(range(1, 1000000))
print(sum_of_squares_optimized(numbers))
```

Output:

```
PS C:\Users\Abhi\Documents\AI Assistant Coding> & C:/Users
333332833333500000
PS C:\Users\Abhi\Documents\AI Assistant Coding>
```

Explanation:

This task analyzes a function that sums squares of numbers and optimizes it using Python's built-in `sum()` with a generator expression. Both versions have O(n) complexity, but the optimized version reduces interpreter overhead and runs faster on large datasets, as shown by successful execution on a million-element list. In practice, execution time improves slightly due to internal optimizations in Python's built-ins. The trade-off is that the optimized version is shorter and faster but slightly less explicit for beginners compared to the loop-based version. This demonstrates how performance gains often come at the cost of reduced step-by-step readability.